

Mini Agent Project — Write-Up

Calculator & Weather Tool Agent (built with Gemini + Python)

1. The ReAct Loop

ReAct stands for Reason, Act, and Observe. In my agent, the model first **reasons** about what it needs to do, then **acts** by calling a tool (calculator or weather_lookup), then **observes** the result that the tool returns. This cycle repeats until the model has enough information to give a **final answer**. For example, when asked 'Which city is warmer, Lahore or Karachi?', the agent called the weather tool twice (once per city), observed both temperatures, and then reasoned that Lahore (35°C) is warmer than Karachi (31°C) before giving the final answer.

2. Tool Schemas Used

Two tools were defined and given to the model using JSON schemas so it knows their name, purpose, and required inputs:

Tool	Inputs	Purpose
calculator	a (number), b (number), operation (add/subtract/multiply/divide)	Performs basic math operations
weather_lookup	city (string)	Returns sample weather data for a given city

Clear descriptions matter because they tell the model exactly what each tool does and what input format it expects. Without a clear description, the model may pick the wrong tool or send the wrong type of argument.

3. Failure Modes Observed

Missing data failure: When asked 'What is the weather in Multan?', the model correctly chose the weather_lookup tool and passed city='Multan'. However, Multan was not part of the sample weather data, so the tool returned 'Weather data not available for this city' instead of a real value. This shows that the tool worked correctly, but the underlying data was incomplete.

Why this matters: A good agent should say clearly that the information is unavailable, instead of guessing or making up a temperature. If the agent invents an answer, the user cannot trust the output.

4. Other Failure Modes & Mitigations (Guardrails)

Failure Mode	Mitigation
Infinite loops	Set a maximum number of loop iterations
Hallucinated tool calls	Only allow tools that are actually defined
Wrong tool arguments	Validate arguments before running the tool
Tool errors / crashes	Catch errors and return a clear message
Missing information	Tell the user data is unavailable, don't guess
Wrong tool selection	Write clear, specific tool descriptions

5. Conclusion

This project built a small agent that uses two tools (calculator and weather lookup) inside a ReAct loop: reason, act, observe, repeat, and finally answer. Testing with a missing city (Multan) showed an important failure mode: the agent must clearly report missing information instead of making up an answer. For simple, single-step tasks, a plain script is enough; agents become useful once a task needs multiple steps, tool use, and decision-making based on results.